# Timed Alignments with Mixed Moves

Thomas Chatain
*dept. name of organization (of Aff.)*
*name of organization (of Aff.)*
City, France
email address or ORCID

Neha Rino
*dept. name of organization (of Aff.)*
*name of organization (of Aff.)*
City, France
email address or ORCID

***Abstract*—The subject of this paper is to study conformance checking for timed models, that is, process models that consider both the sequence of events in a process as well as the timestamps at which each event is recorded. Time-aware process mining is a growing subfield of research, and as tools that seek to discover timing related properties in processes develop, so to does the need for conformance checking techniques that can tackle time constraints and provide insightful quality measures for time-aware process models. In particular, one of the most useful conformance artefacts is the alignment, that is, finding the minimal changes necessary to correct a new observation to conform to a process model. This paper follows a previous one, where we have set our problem of timed alignment. In the present paper, we solve the case where the metrics used to compare timed processes allows mixed moves, i.e. an error on the timestamp of an event may or may not have propagated to its successors, and provide linear time algorithms for distance computation and alignment on models with linear causal processes.**

***Index Terms*—Conformance checking, Alignments, Timestamps, Timed Petri nets**

## I. Introduction

### A. Conformance Checking and Alignments

Process mining studies vast systems through their event logs, and seeks to extract meaningful ways to model the underlying patterns or processes that govern the behaviour of the system, sometimes in order to better understand the system, or as is often the case, predict future behaviour [1]. Once such a process model is obtained, through machine learning or other related techniques, it is natural to ask how one is sure the obtained model is a reasonable approximation of the system's behaviour at all, especially given the lack of explainability in the blackbox approach ML takes to producing solutions. This is where conformance checking comes into the picture, as it is the art of judging the performance of a process model by comparing its behaviour to the observed behaviour of the system, that is, relating modelled and recorded behaviour of a process to each other, independent of the origin of the model [2]. Observed behaviour usually comes in the form the traces of an event log, as a sequence of events that often belong to multiple distinct interleaved instances of processes of the system. On the other hand, we have process models, that were either obtained automatically through mining, or were constructed manually, and act a blueprint to describe what the underlying processes of any given system are supposed to look like. Measures of how well a model reflects system behaviour include fitness, precision, generalisation, and simplicity. We often do not want the system to always precisely generate any and all possible future system behaviour as this can give trivial models accepting for example all execution strings, but on the other hand, neither should they simply regurgitate the event log and accept no new system behaviours, a phenomenon known as overfitting, or not generalising properly. What is often much more useful is a process model that can, up to some small error factor, approximate any reasonable future system behaviour quite closely. This flexible boundary between what is expected and what is possible is precisely what makes a model useful.

In Aryan Adriansyah's seminal thesis [3] , we obtain the notion of an alignment, that is, the minimal series of corrections needed to transform an observed event trace into the execution of the process model that most closely mimics it. It is often seen as an execution of the synchronised product of the process model and a *trace model* ie a simple model that captures exactly one event log trace. This gives us a series of edits, usually insertions or deletions, that can transform a model run into the observed run. For our purposes we will use the term alignment to mean the optimal run of the model that aligns closest to the observation, and how to transform this optimal execution to the observed one will be addressed separately as a matter of distance computation over a set of edit moves. Alignments thereby help pinpoint exactly where inevitable deviations from expected behaviour occur, and the more distant the aligning word of a model is to its observed trace, the worse the model is at reflecting real life behaviour. This can either be used as a tool to help repair the process model to more accurately reflect the observation, or, it can be used to repair the log, or even issue feedback on the behaviour of the underlying system to help it conform to the model better, depending on which components of the operation are the most trustworthy.

### B. Time-aware Processes

Process models are often formalised using the notion of a Petri net, a standard object used to describe distributed systems. They both offer a graphical means by which to represent a complex system, and a formal semantics for their execution, which allows one to mathematically analyse the same. They are useful when trying to express dependencies in seemingly parallel parts of the system, the graph structure allowing for branching (i.e. choice), convergence (i.e. synchronisation), and cycles (i.e iteration) in system behaviour. Assuming the event logs are a list of words over a finite alphabet (the set of possible discrete events), the problem of calculating the alignment has been extensively studied [3] [4]. The notion of distance used on these words used is usually either Hamming distance or Levenshtein's edit distance. Given that Petri net processes already do have certain temporal characteristics, such as a (partial) sense of what happens before and after, and a notion of causal dependency between events, it is natural to want to study explicitly timed systems, that is, systems that not only retain the sequence of events that occur, but the timestamps that show when each event happens. By considering events along with their timestamps when mining processes, observations regarding complex time constraints can be obtained, such as the minimum delay between two events, or the maximum duration the system takes to converge upon a state, or meeting deadlines, all of which are highly relevant in real world applications [5] [6] [7]. In addition, one may want to predict the timestamps of processes [8]. In the community, there are ways to use existing process model notation in order to denote some time constraints. BPMN 2.0 comes equipped with *timer events*, which may be catching or interrupting in nature, and can record absolute, relative, and cyclical time constraints. For our purposes, we use a slightly different formal object whose semantics are capable of expressing both concurrency properties, and the relative and cyclical time constraints that BPMN equipped with timers can. Here, we mean time Petri nets, that is, Petri nets augmented with the ability to record and check the duration it takes to fire a transition once enabled, using which one can impose certain constraints on the relationships between the timestamps of different events.

So far in the study of the alignment problem for process models, we find one notable gap. The process models for which alignment is studied do not consider event timestamps, and constraints imposed upon them. Time-aware process mining seeks to study both what sort of underlying processes govern system behaviour, and what sort of time constraints they can impose on when certain events can occur [9] [10] [11]. The need for such a setting is clear, as specifying and verifying temporal properties, and concrete time constraints regarding the durations of events can provide critical information about a system's behaviour. As time-aware process mining grows popular, new quality measures and conformance checking techniques must be developed that are sensitive to temporal constraints. For this, crucially, one needs to define distance functions that can meaningfully compare and separate different time processes. This paper seeks to provide a framework by which to do the very same.

### C. Three Distances, and Algorithms to Align them

In this paper, we propose three different distance functions over timed words, corresponding to only editing one point in a word at a time (stamp edits), forcing every edit in a word to ripple out and displace all of its causal successors (delay edits), or a combination of the two (mixed edits). The first is essentially Manhattan distance, and hence is extremely practical, and has a quadratic time algorithm in the case of a structurally restricted class of models (linear causal processes), that is, process models whose runs look like straight lines with no branching or concurrency. For all other types of models, we present an encoding in simplex that solves the problem, albeit with exponential worst-time complexity and good efficiency in practice. The second distance function utilises the structure of the process model to study distances between time processes, and we present a straightforward algorithm for solving the alignment in this case, whose time complexity is linear in the size of the causal process and the transition set of the model. This setting may be of slightly limited scope as it is only relevant if one always knows that every single error will have knock-on effects, such as in event logs that only perceive relative durations but output absolute timestamps. The third is the most versatile, and the most interesting for our purposes. In this setting, edits can either be stamp moves or delay moves, allowing one to choose whether to let an edit stay local or affect the rest of the process, depending on which would be cheaper. We present a linear time algorithm for solving the alignment problem in this setting, but once again only for process models with linear causal processes as in the first setting.

## II. Preliminaries

### A. Timed Process Models

We represent events as timed letters, that is of the form $(a, t)$ where $a \in \Sigma$ is the name of the action taken during an event, and $t$ denotes the time at which said action was taken. An event trace is then a sequence of events of the form $\gamma \in (\Sigma \times \mathbb{R}^+)^*$, seen as a timed word. For the context of this paper, we will often ignore the untimed parts of timed words, that is, the projection onto $\Sigma^*$, leaving just the time-series of the timed words, a sequence belonging to $\mathbb{R}^{+*}$.

The timed process model that attempts to capture the behaviour of all observed event traces recorded from the system is represented here as a labelled time Petri net.

**Definition 1** (Time Petri Net)**.** *A labelled time Petri net is a tuple $N = (P, T, F, SI, \lambda, M_0, M_f)$, where*

1) *$P$ is a set of places*
2) *$T$ is a set of transitions*
3) *$P \cap T = \emptyset$*
4) *$F \subseteq (P \times T) \cup (T \times P)$ is a flow relation*
5) *$SI : T \to \mathbb{I}$ is the static interval function, where $SI(t) = (Eft(t), Lft(t))$ such that $Eft$ stands for earliest firing time, and $Lft$ for latest firing time.*
6) *$\lambda : T \to A$ is the labelling function, labelling transitions with actions*
7) *$M_0 : P \to \mathbb{N}$ is the initial marking of $N$*
8) *$M_f : P \to \mathbb{N}$ is the final marking of $N$*

The semantics of a time Petri net are a bit involved. Given a transition $t \in T$ we define the pre-set of $t$ as ${}^\bullet t = \{p \in P | (p, t) \in F\}$ and its post-set similarly is defined as $t^\bullet = \{p \in P | (t, p) \in F\}$, just as it is defined for untimed Petri nets (the presets and post-sets of places are defined similarly). A transition $t$ of a time Petri net is *enabled* at marking $M$ iff $\forall p \in {}^\bullet t : M(p) > 0$. The set of all enabled transitions at a marking $M$ is denoted by *Enabled(M)*.

This formalism begins to differ from untimed Petri nets with the notion of states and clock functions. The current marking is not sufficient to define the state of the whole system, as the time elapsed since a transition was enabled is crucial information to define the execution of the system. Hence, a *state* of a time Petri net $N = (P, T, F, SI, M_0)$ is a pair $S = (M, I)$, where $M$ is a marking of $N$ and $I : Enabled(M) \to \mathbb{R}^+$ is called the *clock function*. The initial state is therefore $(M_0, \mathbf{0})$, where $\mathbf{0}$ is the zero function.

A transition $t$ is fireable from state $S = (M, I)$ after delay $\theta \in \mathbb{T}$ iff $t$ is enabled at $M$, and updating the clock function to increment by delay $\theta$ will keep $t$'s new clock value $I(t) + \theta$ in the range $[Eft(t), Lft(t)]$ as determined by the static interval function. This is also referred to as $t$ firing at time $I(t) + \theta$ at state $(M, I)$.

Once $t$ is fired, the marking is updated to $M'$ as in the untimed case (subtracting one token from each preplace, and adding one token to each postplace), and the clock function $I$ is updated to $I'$ by increasing clock values by $\theta$ for any transition that was enabled at both $M$ and $M'$, setting clock values to 0 for any transitions newly enabled by $M'$ that were not enabled at $M$, and leaving clock values undefined for any transitions that are not currently enabled. This is denoted by $(M, I)[t\rangle(M', I')$

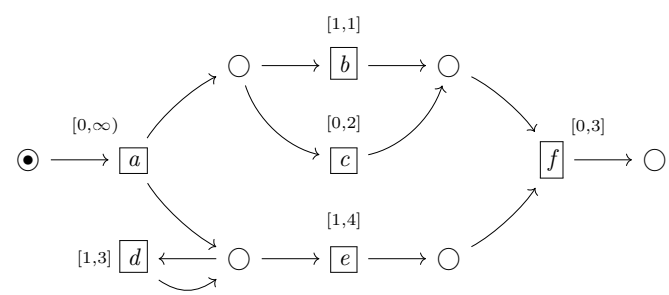
An important feature of time Petri nets is the notion of urgency, that is, if a transition has been enabled then it must either be disabled (lose some tokens from its preset due to another transition firing) or be fired before the latest firing time lapses. It cannot be allowed to simply *time-out*, i.e, waiting, unfired, throughout the period of time is it capable of firing, and then being disabled just by having the Lft lapse, that is if $t$ is enabled at marking $M$ and has clock value $Lft(t)$, it must fire, or another transition must fire at the same instant disabling $t$.

Now, we want a notion of a valid execution of the model, that is, process traces. This is why we equip our time Petri nets with a notion of a final state, or in this case, a final marking. A valid execution begins at the initial marking, and fires a sequence of transitions (representing a series of activities occuring at certain times) and the updated marking at the end of the firing sequence being $M_f$, with any clock function $I$.

**Definition 2** (Language of a time Petri net)**.** *A word $w = (a_0, a_1, \dots a_n) \in A^*$ is in the language of the labelled time Petri net $\mathcal{L}(N)$ if there is a firable sequence of transitions $(t_0, t_1 \dots t_n) \in T^*$ such that $\lambda((t_0, t_1, \dots, t_n)) = w$ and they transform the initial marking into the final one, that is, for some clock function $I$ on $M_f$,*

$$(M_0, \mathbf{0})[t_0, t_1, \dots t_n\rangle(M_f, I)$$

**Example 1.** *Consider the following example of a time Petri net $N$:*




*One possible execution of $N$ would be for the firing sequence*

$$w = (a, 1)(b, 2)(d, 3)(e, 4)(f, 5)$$

*The initial marking only has $a$ enabled, and firing $a$ at time 1 updates the marking by removing the token from $a$'s preplace and filling its two post-places, thereby enabling $b, c, d, e$. Now, note by urgency that $c$ can never fire past time 2 as then $b$ would have already fired by urgency, but in this execution this doesn't matter as $b$ fires at 2 regardless, filling one preplace of $f$. Then $d$ fires, having been enabled for 2 units of time, thereby resetting the enabling time of both $d$ and $e$ to zero, so that when $e$ does fire it has only been most recently enabled for 1 unit of time. Now, finally, $f$ is enabled, and fires after 1 unit of having been enabled.*

## III. The Alignment Problem in Timed Settings

The alignment problem, given a trace from an event log and a process model, involves finding a valid execution of the process model that is closest to the trace under some metric.

**Definition 3** (The General Alignment Problem)**.** *Given a process model $N$ denoted by a Time Petri Net and an observed timed trace $\sigma$ we wish to find a timed word $\gamma \in \mathcal{L}(N)$ such that*

$$d(\sigma, \gamma) = \min_{x \in \mathcal{L}(N)} d(\sigma, x)$$

*for some distance function $d$ on timed words.*

In the untimed setting, this is viewed as a problem of minimizing cost over a series of edit moves, either insertions (where the model reads a letter that wasn't in $\sigma$) or deletions (where $\sigma$ has a letter that the model does not wish to read). When aligning timed words, clearly there are two crucial aspects to the problem, which are, how one deals with deviations in the action labels themselves (the untimed letters), versus how one deals with deviations in just the timing properties of the word (the timestamps). The alignment problem has been extensively studied for the untimed case, and so, if we simply sought to align the action labels of the word to the underlying untimed Petri net, this is a solved problem. On the other hand, timestamps are attached to the letters they belong to, so in a sense, performing task A at time $t_1$ as compared to performing task B at time $t_2$ is not easily comparable, as the time constraints on different tasks are often different. In this manner, it is not clear how meaningful it would be to define edit moves that transform timestamps that are attached to different tasks.

This is why, we first restrict our attention to the case where the untimed part of $\sigma$ does match the model, that is to say, if $\sigma = (w, \tau)$ where $w$ is the untimed letter sequence and $\tau$ is the timestamp sequence. then we assume that $w$ has a valid causal process $(CN, p)$ over $N$. Another restriction we assume is that all of the process models we study are linear in shape, which allows us to depict timing functions as linear words. It should be noted that our techniques work for a slightly larger class of models, see section II B of <paper1> for more details. Clearly this degenerate case of the problem needs to be solved if the general timed alignment problem is to be solved, and we proceed to show that this case is interesting and complex in its own right.

This gives us a slightly different problem than the general setting of the alignment problem, that focuses purely on timing properties, that is, a purely timed alignment problem, as defined below :

**Definition 4** (The Purely Timed Alignment Problem)**.** *Given a process model $N$ denoted by a Time Petri Net and an observed timed trace $(w, \sigma)$, such that $w \in Untime(\mathcal{L}(N))$ we wish to find a valid timestamp sequence $\gamma$ such that*

$$d(\sigma, \gamma) = \min_{x \in \mathcal{L}(N)} d(\sigma, x)$$

*for some distance function $d$ on timestamp sequences.*

Hence, for the first few sections of this paper, we will proceed to ignore the untimed parts of the words, i.e we assume the word $\sigma$ we wish to align is just a sequence of timestamps, and simply seek to find timing functions on said causal process that are both valid, and minimize distance to $\sigma$ under the metric of choice.

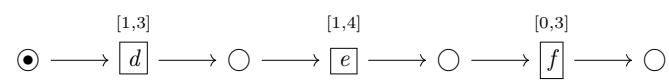
We shall revisit the general timed alignment problem in section V, and show how our methods for solving the purely timed alignment problem can be adapted using existing techniques to provide approaches for solving the general timed alignment problem.

### *A. Edit Moves for Timed Words*

Now we come to the problem of deciding how to compare two timing functions over the same causal process, and quantify how close they are to each other.

Much like Levenshtein's edit distance, popularly used in the untimed case of the alignment problem, we view the definition of these distances as an exercise in cost minimisation over the set of all transformations between two words. In order to formalise the same, we need to define what the valid moves of such a transformation could be. We define *moves* as functions that map one time sequence to another. What sort of functions are useful notions of transformation on a timed system?

**Example 2.** *Let us look at a model that is a single branch of example 1, and study the process model $N_1$ presented here:*




*Now, say we had the following words that did not fit the model, and we wished to analyse how best to modify them to fit them back into the model.*

*We start with $(u_1, \sigma_1) = (def)(4, 6, 6)$ A reasonable candidate for the closest valid execution would be if the timing function $\sigma_1$ were replaced by $\gamma = (3, 6, 6)$, and it feels reasonable to say that the cost for aligning $\sigma_1$ to this $\gamma$ is 1. A way to arrive at this conclusion is by trying to execute $\sigma$'s firing sequence, where we see that only the guard for $d$ fails. If $d$ were just shifted to fire at 3 instead, the whole run would execute without a hitch. This sort of local, almost typographical error can often happen in systems, and it is the simplest kind to fix.*

*We start with $(u_1, \sigma_2) = (def)(4, 8, 11)$ The closest valid timing function would try to preserve firing times, but the moment it tries to fire $d$, $\sigma_2$ runs late throughout. The closest we can get, intuitively, is to fire every transition in the model as late as possible, giving $\gamma = (3, 7, 10)$. Now, when trying to compare these firing sequences, we can of course view it just like we did for $\sigma_1$, as $d$, $e$ and $f$ all fire later than they should, each timestamp is moved back once, giving an aligning cost of 3. There is however, another way to see this deviation. This cascading chain of errors can be fixed if $d$ is moved back to fire at 4, and all the relative relationships between $d$ and its successors are preserved. This views the tasks $e$ and $f$ as only caring about when $d$ ended, which makes sense, because they are causally linked and hence do only start once their predecessor ends. This*

*means, the switch from the timestamp series* $(4, 8, 11)$ *to* $(3, 7, 10)$ *can be viewed as only a cost 1 edit, if we assume* $e$ *and* $f$ *only care about the relative durations or delays between activities. This is a slightly more complex error to conceive of, but it is of natural practical use, if a delay at the beginning caused the whole process to occur too late, the important thing to fix is just the delay at the beginning, and one can sometimes assume with that sorted, the rest of the process will now conform to the model as needed.*

Based on the above example, we naturally arrive at two types of moves.

We define a *stamp move* as a move that translates the timing function only at a point, ie, that edits a particular element of the timestamp series $\tau$. Calculating $d_t$ between two traces can be viewed as a cost minimization process in aligning the traces using only stamp moves.

**Definition 5** (Stamp Move)**.** *Given a timing function* $\gamma : \{1, \dots n\} \to \mathbb{R}$*, formally, we define this as :*
$\forall x \in \mathbb{R}, i \leq n : stamp(\gamma, x, i) = \gamma'$ *where*

$$\forall i \leq n : \gamma'(j) = \begin{cases} \gamma(j) + x & j = i \\ \gamma(j) & otherwise \end{cases}$$

The next type of move we describe is the more novel and interesting *delay move*. By formulating this type of edit move, we sought to leverage the structure of the process model itself, by reflecting the causal relationships between events.

A stamp move is purely local, such that when a stamp edit occurs at $e$ its immediate successor events shift their relationship with $e$, thereby ensuring that the change in $e$'s timestamp does not derail the rest of the system. On the other hand, any delay move at $e$ will preserve relative relationships in the future, at the cost of shifting the timestamp of every causal descendent of $e$ by the same amount.

This brings us to the following definition :

**Definition 6** (Delay Move)**.** *Given a timing function* $\gamma : \{1, \dots n\} \to \mathbb{R}$*, formally, we define this as :*
$\forall x \in \mathbb{R}, i \leq n : stamp(\gamma, x, i) = \gamma'$ *where*

$$\forall i \leq n : \gamma'(j) = \begin{cases} \gamma(j) + x & j \geq i \\ \gamma(j) & otherwise \end{cases}$$

Now, armed with these types of moves, three natural notions of distance can be constructed.

**Definition 7** (Stamp Only Distance : $d_t$)**.** *Given any two timing functions* $\tau_1, \tau_2$ *over the same causal process* $(CN, p)$*, we define the stamp-only distance* $d_t$ *as follows :*

$$d_t(\tau_1, \tau_2) = \min\{cost(m) | m \in Stamp^*, m(\tau_1) = \tau_2\}$$

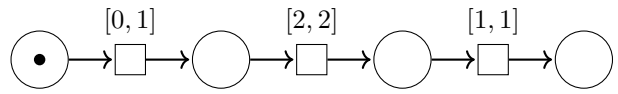
**Definition 8** (Delay Only Distance : $d_\theta$)**.** *Given any two timing functions* $\tau_1, \tau_2$ *over the same causal process* $(CN, p)$*, we define the delay-only distance* $d_\theta$ *as follows :*

$$d_t(\tau_1, \tau_2) = \min\{cost(m) | m \in Delay^*, m(\tau_1) = \tau_2\}$$

**Definition 9** (Mixed Moves Distance : $d_N$)**.** *And thirdly, given any two timing functions* $\tau_1, \tau_2$ *over the same causal process* $(CN, p)$*, we define the mixed move distance* $d_N$ *as follows :*

$$d_t(\tau_1, \tau_2) = \min\{cost(m) | m \in (Stamp \cup Delay)^*, m(\tau_1) = \tau_2\}$$

**Example 3.** *Consider the following example :*




*Now for this* $N$*, let the observed trace* $\sigma = (0, 3, 4) \notin \mathcal{L}(N)$*.*

*The best* $d_t$ *alignment for the example in the diagram below is* $\gamma_t = (1, 3, 4)$ *because*

$$d_t(\gamma_t, \sigma) = 1 \leq d_t((x, x{+}2, x{+}3), \sigma) = |x| + 2|x - 1|, \forall x \in [0, 1]$$

*The best* $d_\theta$ *alignment for the above trace and model pair is* $\gamma_\theta = (0, 2, 3)$ *because*

$$d_\theta(\gamma_\theta, \sigma) = 1 \leq d_\theta((x, x + 2, x + 3), \sigma) = |x| + 1, \forall x \in [0, 1]$$

*Now we consider the mixed setting. Once again, we note that any word in the language would be of the form* $(x, x + 2, x + 3)$ *with* $x \in [0, 1]$*.*

*Now intuitively, when dealing with linear models where the ripple effect of a delay is always predictably to the right irrespective of the values of the timestamps, moves can be applied in any order, so we can assume a minimum cost move is chronological. So our first move will have to incur a minimum cost of* $|x| = x$ *as we try to align* $(0, 3, 4)$ *to our model. Now, if any part of this initial move was a delay, we would push the second component even further away from* $x + 2$ *than it already was, and if any of the delay were negative, then the mixed move seems counterproductive at the first component as the delay and stamp moves work against each other, so let us say the first move was pure stamp. Now we move on to move two, which will again incur a minimum cost of* $|1 - x| = 1 - x$*. Now, if the stamp part of this move were positive, it would leave the third component further off from the goal* $x + 3$ *than if all of the move was delay, and so a pure delay move seems best here. Hence, the least cost alignment is* $(x, 0, 1)(0, 1 - x, 2)$*, with total cost 1.*

*All the intuitive reasoning given above will be justified in subsequent sections, and in fact distance 1 is the best we can do here even in the mixed case. Hence the aligning words here constitute the whole language* $\mathcal{L}(N)$ *as they are all at distance 1 from the observed trace in this setting.*

**Example 4.** *Consider the same underlying linear model, and a new observed trace* $\sigma = (3, 4, 5) \notin \mathcal{L}(N)$*.*

*The best* $d_t$ *alignment for the example in the diagram below is* $\gamma = (1, 3, 4)$ *with minimum cost* $d_t(\sigma, \gamma) = 4$*.*

*The best* $d_t = \theta$ *alignment for the example in the diagram below is also* $\gamma = (1, 3, 4)$*, but this time with*

*minimum cost* $d_\theta(\sigma,\gamma) = 3$*, evidenced by the move sequence* $(delay(-2,1)delay(+1,2))$*.*

*And lastly, the best* $d_N$ *alignment for the example in the diagram below is also* $\gamma = (1,3,4)$*, the sequence of moves being simply one stamp and one delay move at the start,* $m = stamp(-1,1)delay(-1,1)$*, and now with minimum cost* $d_N(\sigma,\gamma) = 2 < \min\{d_t(\sigma,\gamma), d_\theta(\sigma,\gamma)\}$*.*

## IV. Results and Algorithm : Mixed Setting

Given the nonconstructive nature of the definition of $d_N$, it is not clear how one can efficiently calculate the distance between two fixed timed traces, as a minimal cost sequence of moves is not obvious. As before, we first study the problem over linear causal processes, as these make analysing the effects of moves significantly simpler. Before we propose an algorithm that does calculate minimal cost for linear timed traces, we first define a few properties that seem to characterise classes of well formed minimal cost runs, prove that $d_N$ is well defined along the way, and then proceed to prove that these properties both improve cost, and are satisfied solely by the run calculated by the algorithm we provide.

We start with some convenient notation for a common combination of the previously defined moves.

**Definition 10** (Mixed Move)**.** *We define mixed moves, that denote doing a stamp move and a delay move at the same position in the word. We define their effect as follows :*

$$(s,d,e)(\gamma) = stamp(delay(\gamma,d,e),s,e)$$

*Let the set of all mixed moves be* $Moves$*.*

Given any move $m \in Moves$ we define the function $cost : Moves \to \mathbb{R}^+$ that returns the cost of the move. The cost of a mixed move $(s,d,e)$ is the sum of the cost of the stamp and delay moves it's made up of, $|s| + |d|$.

We say a sequence of moves $m_1 m_2 \dots m_n = m \in Moves^*$ *aligns* $\tau_1$ to $\tau_2$ if

$$m(\tau_1) = m_n(\dots(m_2(m_1(\tau_1))\dots) = \tau_2$$

With delay edits, which constitute a new way of thinking about how a time-series can be transformed, comes a new perspective with which we can view timing functions over causal processes. There is of course the standard definition, $\tau : \{1,\dots n\} \to \mathbb{R}^+$ that assigns to each event a timestamp that records exactly when the event occurs.

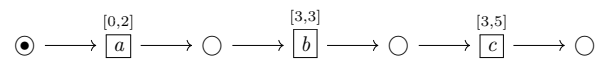
Instead, thinking along the lines of delays and durations between events occuring, we will often benefit from considering the following representation when speaking about delay moves, and so we define a way by which to view a timed word not in terms of its absolute timestamps, but by the delays between relevant timestamps.

**Definition 11** (Flow Function)**.** *Given a (not necessarily valid) time sequence,* $\tau : \{1,\dots n\} \to \mathbb{R}^+$*, we first define the flow function of* $\tau$*,* $f_\tau : \{1,\dots n\} \to \mathbb{R}^+$ *such that*

$$f_\tau(i) = \begin{cases} \tau(i) & i = 1 \\ \tau(i) - \tau(i-1) & i > 1 \end{cases}$$

The flow function thereby is a dual representation of timing functions, much like the relationship between graphs and line graphs, as if a timing function traditionally labels its transition nodes with timestamps, the flow function labels edges leading up to transitions with the duration since said transition was enabled.

**Example 5.** *Consider the following process model* $N_2$*:*




*The word* $w = (abc)(1,5,9)$ *is not a member of* $\mathcal{L}(N_2)$ *but its untimed part belongs to the language of the untimed underlying net. For this net and word, we can confuse transition labels with event labels, so we can define its flow function as* $(1,4,4)$*.*

As defined, $f_\tau$ as defined produces exactly the time durations that the guards of each transition in the model checks, ie the clock function values during the run. Hence, we see that if a word is in the language of the model, then its $f$ function maps events to values that lie within the constraint that the event's corresponding transition demands, that is, it is perfectly aligned with the model with distance 0. This condition is unfortunately not sufficient though, due to urgency concerns, but it is quite close to the exact condition necessary for a word to be in the language of a time Petri net.

Also note that given the underlying causal process and the resulting $f_\tau$, we can reconstruct $\tau$ quite straightforwardly as

$$\forall i \le n : \tau(i) = \sum_{j \le i} f_\tau(j)$$

With a new representation, it is natural to study how the transformations we have defined earlier on timing functions transform their respective flow functions. A stamp move at an event $e$ of value $+x$ will increase the duration between $e$ and its immediate enabling predecessor, while shrinking the duration between $e$ and its immediate enabled successor by an equal amount, thereby moving $e$ by $x$ and compensating appropriately so none of the future timestamps change at all. On the other hand, a delay move at $e$ will increase the duration between $e$ and its immediate enabling predecessor, keeping all other durations constant, thereby ensuring $e$ and all of its causal descendents have their timestamps shifted by $x$. So both these moves are much more local as seen by the flow function, effectively perturbing only the duration attached

to $e$ or its immediate enabled successor, while in the timing function representation moves at a position could have consequences well beyond its immediate neighbours. This gives us the following lemma :

**Lemma 1.** *Given a flow function $f_{\tau_1}$ and a mixed move $(s, d, i)$, the consequence of performing the move is the flow function defined below, where $\tau_2 = (s, d, i)\tau_1$.*

$$f_{\tau_2}(j) = \begin{cases} f_{\tau_1}(j) + s + d & j = i \\ f_{\tau_1}(j) - s & j = i - 1 \end{cases}$$

We wish to cull down the space of possible sequences of moves to smaller, equivalent spaces that still capture all the behaviours we're interested in, while being a little more well behaved. Hence, we introduce the following properties, which are designed intuitively to be sequences of moves that still perform the same effective transformations, and generally lower the cost of the transformation too by choosing moves wisely to avoid inefficiency. These properties (order, co-operation, and stability) have been chosen specifically to uniquely characterise the runs of the algorithm we devise to compute $d_N$ on pairs of linear traces.

### A. Order

The first property captures the order in which moves in a sequence target the positions in a linear trace. It would be good to be able to focus our attention on sequences of moves that proceed in an orderly fashion, either from left to right, doing exactly one mixed move at each position. This is intuitively reasonable as doing multiple mixed moves at the same position can only worsen the cost if it at all changes it, and assuming that mixed moves commute on linear words, it would be all the same to arrange them in order of the position it takes effect at. This gives us the following two properties.

We try to follow through the following properties in both the timing function and the flow function.

**Definition 12** (Chronology)**.** *A sequence of moves aligning $\gamma$ to $\sigma$ is said to be chronological if for all positions $i < j \leq n = |\gamma|$, all the moves at position $i$ are performed before any move at position $j$ and exactly one mixed move (where one or both components may be zero) takes place at each position, that is $\rho \in Moves^*$ is chronological iff*

$$\forall i \in \{1, 2, \ldots n\}, \exists s_i, d_i \in \mathbb{R} :$$
$$\rho = (s_1, d_1, 1)(s_2, d_2, 2) \ldots (s_n, d_n, n)$$

*Such that*

$$\forall i \in \{1, 2, \ldots n\} : \sigma_i = \gamma_i + s_i + \sum_{j=1}^{i} d_j$$

**Definition 13** (Reverse Chronology)**.** *A sequence of moves aligning $\gamma$ to $\sigma$ is said to be reverse chronological if for all positions $i < j \leq n = |\gamma|$, all the moves at position $i$ are performed after any move at position $j$ and exactly one mixed move (where one or both components may be zero) takes place at each position, that is $\rho \in Moves^*$ is chronological iff*

$$\forall i \in \{1, 2, \ldots n\}, \exists s_i, d_i \in \mathbb{R} :$$
$$\rho = (s_n, d_n, n)(s_{n-1}, d_{n-1}, n-1) \ldots (s_1, d_1, 1)$$

*Such that*

$$\forall i \in \{1, 2, \ldots n\} : \sigma_i = \gamma_i + s_i + \sum_{j=1}^{i} d_j$$

**Definition 14** (Ordered)**.** *A sequence of moves aligning $\gamma$ to $\sigma$ is said to be ordered if it is either chronological or reverse chronological.*

Clearly, all chronological sequences have a one to one correspondence with reverse chronological sequences with exactly the same cost, the map between the two being simply reversing the sequence of moves.

In addition, in the flow function representation, the notions of chronology and reverse chronology are still somewhat preserved. A mixed move affects both the duration before and all those after the event the move is applied at, so the order in which they happen does indeed stay chronological or reverse chronological as the case may be. The only point of nuance here is that of course, a prefix of a sequence of say chronological moves is no longer has the property of having completely aligned the word up to the i-th element of the flow vector while leaving the rest unchanged, as the last edit may have perturbed the (i+1)st element.

But, the upshot is, in a reverse chronological sequence of moves, the prefix of the sequence that stops at the ith event, combined with the stamp component of the mixed move at position (i-1), completely aligns the flow function for the suffix starting at position i. Reverse chronological runs do not have an equivalent partial completion property in the timing function setting, but due to how local the moves end up being in the flow function setting, we obtain it here.

**Note :** For simplicity, we always assume the move played on the last position of a trace henceforth is a pure delay. Even if part of it were stamp, it would not affect the flow or timing representation any differently, so this is safe to assume.

**Example 6.** *Given $\gamma = (1, 1, 2, 4, 5)$ and $\sigma = (1, 2, 2.5, 4.2, 5)$, an example of a cost 5.3 non-chronological sequence of moves aligning them is as shown below.*

$$\gamma \xmapsto{(-1,0,1)} (0, 1, 2, 4, 5) \xmapsto{(0,2,1)} (2, 3, 4, 6, 7) \xmapsto{(0,-1,1)}$$
$$(1, 2, 3, 5, 6) \xmapsto{(0.3,-0.8,3)} (1, 2, 2.5, 4.2, 5.2) \xmapsto{(0,-0.2,5)} \sigma$$

*There is an improved cost 3.3 chronological sequence of moves obtained by reordering the above sequence and combining moves at the same position :*

$$\gamma \xmapsto{(-1,1,1)} (1, 2, 3, 5, 6) \xmapsto{(0,0,2)} (1, 2, 3, 5, 6) \xmapsto{(0.3,-0.8,3)}$$

$$(1,2,2.5,4.2,5.2) \xmapsto{(0,0,4)} (1,2,2.5,4.2,5.2) \xmapsto{(0,-0.2,5)} \sigma$$

*There corresponding reverse chronological sequence of moves is of course :*

$$\gamma \xmapsto{(0,-0.2,5)} (1,1,2,4,4.8) \xmapsto{(0,0,4)} (1,1,2,4,4.8) \xmapsto{(0.3,-0.8,3)}$$

$$(1,1,1.5,3.2,4) \xmapsto{(0,0,2)} (1,1,1.5,3.2,4) \xmapsto{(-1,1,1)} \sigma$$

As the above example suggests, we claim the following :

**Lemma 2.** *There is a minimal cost sequence of moves aligning any two linear timed traces that is chronological/reverse chronological.*

*B. Co-operation*

**Definition 15** (Co-operation)**.** *A mixed move is said to be co-operative if its delay edit and stamp edit are in the same direction, that is,*

$$(s,d,i) \text{ is co-operative iff } sd \geq 0$$

*An ordered sequence of moves $m = m_0 m_1 \dots m_{n-1}$ aligning $\gamma$ to $\sigma$ is said to be co-operative if each of its moves is co-operative.*

This property has nothing to do with the words it is trying to align, and as a result, is largely unaffected by whether we're viewing the words in timing or flow representation.

**Example 7.** *The previous example (that was a cost 3.3 run, recall) can hence be further improved to become co-operative cost 1.7 run as follows :*

$$\gamma \xmapsto{(0,0,1)} (1,1,2,4,5) \xmapsto{(1,0,2)} (1,2,2,4,5) \xmapsto{(0.5,0,3)}$$

$$(1,2,2.5,4,5) \xmapsto{(0.2,0,4)} (1,2,2.5,4.2,5) \xmapsto{(0,0,5)} \sigma$$

*With a reverse chronological counterpart on the flow vector:*

$$f_\gamma \xmapsto{(0,0,5)} (1,0,1,2,1) \xmapsto{(0.2,0,4)} (1,0,1,2.2,0.8) \xmapsto{(0.5,0,3)}$$

$$(1,0,1.5,1.7,0.8) \xmapsto{(1,0,2)} (1,1,1.5,1.7,0.8) \xmapsto{(0,0,1)} f_\sigma$$

Once again, as the example suggests, one can convert a non-cooperative run into a cooperative one and improve cost while we're at it.

**Lemma 3.** *There is a cooperative minimal cost chronological (or reverse chronological) sequence of moves aligning any two linear timed traces.*

Very similar to co-operation is the notion of *cross co-operation*. Much like how mixed moves at a position should work together instead of against each other, a stamp move $s$ at a location should ideally have opposite sign to the delay $d$ at the next position, as its contribution to the next flow component is going to be $-s$. This leads us to the following definition :

**Definition 16.** *A reverse chronological run $m = (0,d_n,n)\dots(s_1,d_1,1)$ is said to be cross co-operative if no stamp component has the same sign as the delay component of the move played at the next position, that is,*

$$\forall i < n : s_i \cdot d_{i+1} \leq 0$$

Much like co-operation above, cross co-operation can be used to improve the cost of a run, as seen in the next lemma.

**Lemma 4.** *For every reverse chronological co-operative run $m'$ aligning $\gamma$ to $\sigma$, there is a reverse chronological, co-operative and cross co-operative run $m$ aligning $\gamma$ to $\sigma$ such that*

$$cost(m) \leq cost(m')$$

*C. Stability*

The final property we define will be largely used for reverse chronological runs on flow function representations, and it is the last improvement one can make on the cost of a sequence of moves as after enforcing this property, the sequence of moves that results is unique. We stop to prefer this representation for our current needs for two reasons, one is that the reverse chronological setting is more useful for other types of process models where chronological algorithms fail, and the insistence on the flow function representation is because when dealing with reverse chronological models, the flow function still retains the notion of partially aligning suffixes of the traces as we go along, which is quite hard to see as clearly in the timing function representation. Moreover, we define stability in terms of flow values rather than timestamp values because flow vectors and the effects of moves on them can be studied locally, while timestamp series get perturbed throughout their length by individual moves, and hence trying to optimise for the effect of a single mixed move involves too many variables. Hence, until further notice, consider any co-operative run to be reverse chronological, and definitions and algorithms will favour aligning flow functions rather than timing functions. It is quite clear to see that completely aligning a flow function is equivalent to completely aligning its timing function, so this is acceptable.

The notion of co-operation and reverse chronology leaves only one degree of freedom so to speak in our choice of aligning moves. One say, the stamp components of a sequence of moves has been decided, the delay components are predetermined to align the flow function from right to left.

Now, at any point, a delay move only affects one flow component, and adds to the cost of one mixed move. A stamp move also only adds to the cost of one mixed move, but it has the ability to affect two flow components, hence it is natural to want to choose stamp moves in a manner that corrects both the flow components it belongs to to

the best of its abilities, thereby extracting as much use as possible from as little cost, and leaving the rest up to the delay component to correct. It is this reasoning that brings us to the property we define below as stability, which denotes this prudent choice of $s$.

**Definition 17** (Stability)**.** *In a reverse chronological, co-operative sequence of moves, let*

$m_i = (s, d, i)$ *be a co-operative move seeking to correct the partially aligned flow function*

$f_\gamma = (f_\gamma(1), \dots f_\gamma(i), f_\gamma(i+1), f_\sigma(i+2) \dots f_\sigma(n))$ *(the result of the run of $m$ all the way up to the last stamp move) to* $f_\sigma = (f_\sigma(1), f_\sigma(2), \dots f_\sigma(n))$.

*Let* $e_i = f_\sigma(i) - f_\gamma(i)$, *and* $e_{i+1} = f_\sigma(i+1) - f_\gamma(i+1)$.

*We say $m_i$ is stable if*

$$s = \begin{cases} 0 & e_i \cdot e_{i+1} \geq 0 \\ e_i & \textit{Else If } |e_i| < |e_{i+1}| \\ -e_{i+1} & \textit{Otherwise} \end{cases}$$

By default that the only stable move at the last position is a pure delay.

A co-operative sequence of moves is said to be *stable* if each of its moves is stable.

Note that any stable run is also cross co-operative.

**Lemma 5.** *The stable, co-operative, reverse chronological sequence of moves aligning any two linear timed traces is unique.*

*Proof.* This is clear, as given a partially aligned flow trace the choice of $s$ to make the next move stable as defined above is deterministic, and the corresponding delay moves are determined by reverse chronology and co-operation. □

**Example 8.** *Hence, the earlier cost 1.7 example can be even further improved to be stable and have cost 1.5, thereby giving us the following :*

$$f_\gamma \xmapsto{(0,-0.2,5)} (1,0,1,2,0.8) \xmapsto{(0,-0.3,4)} (1,0,1,1.7,0.8)$$
$$\xmapsto{(0,0,3)} (1,0,1,1.7,0.8) \xmapsto{(0.5,0.5,2)} (1,1,1.5,1.7,0.8) \xmapsto{(0,0,1)} \sigma$$

Once again, as the example suggests, we claim that stability improves the cost of the sequence of moves, but moreover, a stable chronological run (or reverse chronological run) is unique.

**Lemma 6.** *There is a stable minimal cost sequence of moves aligning any two linear timed traces.*

### *D. Computing $d_N$ and Alignments*

**Lemma 7.** *Given two time sequences* $\sigma = (\sigma_1, \sigma_2, \dots \sigma_n)$ *and* $\gamma = (\gamma_1, \gamma_2, \dots, \gamma_n)$ *with linear underlying causal processes, the sequence of moves the above algorithm calculates (m) corresponds to the unique stable sequence of moves that aligns $\sigma$ to $\gamma$.*

*Proof.* This is clear by the definition of stability. □

**Algorithm 1** $d_N$ Backwards Computation Algorithm

```
Input :  σ, γ
Output :  d_N(γ, σ)
cost ← 0
i ← n
while i > 1 do
    a ← f_σ(i) − f_γ(i)
    b ← f_σ(i − 1) − f_γ(i − 1)
    if a · b ≥ 0 then                          ▷ (0, a, i)
        γ ← (0, a, i)γ
    else if |a| < |b| then                     ▷ (−a, 0, i − 1)(0, 0, i)
        γ ← (−a, 0, i − 1)γ
    else                                       ▷ (b, 0, i − 1)(0, a − b, i)
        γ ← (b, 0, i − 1)(0, a − b, i)γ
    end if
    cost ← cost + |a|
    i ← i − 1
end while
cost ← cost + |γ_1 − σ_1|                      ▷ (0, |σ_1 − γ_1|, 1)
```

**Theorem 8.** *Algorithm 1 is correct, that is, its result (cost)* $= d_N(\gamma, \sigma)$

*Proof.* This holds as by Lemma 7 the algorithm calculates the unique stable sequence of aligning moves, and by Lemma 6 this must be the minimal cost sequence. □

**Theorem 9.** *Given a linear causal process $(CN, p)$ of a time Petri Net $N$ and a linear observed trace $\sigma$, the word $\gamma \in \mathcal{L}(N)$ such that*

$$f_\gamma(i) = \operatorname*{argmin}_{x \in [a_i, b_i]} |x - f_\sigma(i)|$$

*also has the property* $\gamma = \operatorname*{argmin}_{x \in \mathcal{L}(N)} d_N(x, \sigma)$.

This of course means that we have a linear time algorithm for aligning linear causal processes under $d_N$, by locally choosing the best flow vector as defined by the formula

$$f_\gamma(i) = \operatorname*{argmin}_{x \in [a_i, b_i]} |x - f_\sigma(i)|$$

This, incidentally, is exactly the aligning word obtained by the algorithm developed for delay-only distance, $d_\theta$ in <paper 1>.

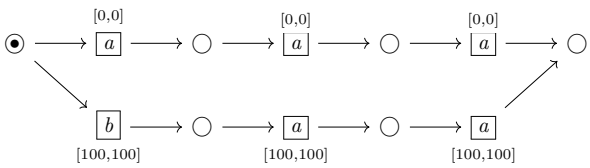
## V. The General Timed Alignment Problem

Now that we have provided some ways to tackle the purely timed alignment problem in a number of metrics and settings, we come back to the general timed alignment problem. One simple approach for solving the general timed alignment problem would be simply to ignore the timed aspect completely at the start, viewing only the untimed part of the word $w$ and the process model $N$ (now reduced to the underlying untimed Petri net)

and applying the $A^*$ algorithm, or any other untimed alignment approach. This will produce an untimed word in the language of the untimed model, that is, a word $w'$ with a valid causal process. This new untimed word $w'$ can then be given a timing function based on that of $w$, preserving time for substitution edits, synchronous moves, and repeating or deleting timestamps for any pure insertion or deletion moves. This now leaves us with a valid causal process and a potentially invalid timing function over it, which can then be cast as a purely timed alignment problem.

This approach is quite naive, as it may have a tendency to exaggerate the importance of action label errors. For example, in the following scenario, perhaps the second alignment is better than the first, but it would be overlooked by the approach we described above.

**Example 9.** *Consider the following net $N_3$ where the final marking is presumed to be the sink place:*




*Now, let the word we seek to align be $w = (a, 100)(a, 100)(a, 100)$.*

*The alignment the approach we described above would provide is $w' = (a, 0)(a, 0)(a, 0)$.*

*But if we assign even a tenth of the cost of action edits to timestamp edits, clearly the closer word in the language was $w'' = (b, 100)(a, 100)(a, 100)$.*

This leaves us once again with the question of how to balance the tradeoff between timestamp edits and action label edits. A more balanced approach would assign a particular cost $c_A$ to action edits and another $c_T$ to time edits, and devise an algorithm to minimise their sum. A potential fix to our previous idea would be to essentially retain the above approach, but instead of just doing it for the best untimed alignment, take a selection of good untimed alignments (up to some allowable threshold), and try to align their timestamps for each untimed candidate, and then minimise the total cost over this set. This alleviates some of the bias towards preserving action labels by allowing for action deviations in case the timestamp alignment proves particularly easy, but this is still just an initial approach to the wider problem, and we hope to find better methods in the future.

## VI. Implementation

We have implemented the $d_N$ computation algorithm and the $d_N$ alignment algorithm for linear process models in python, available at <link>. Both algorithms have linear time complexity, and as a result, they have efficient running times, as evidenced by the table below :

## VII. Perspectives and Conclusion

## References


[1] W. M. P. van der Aalst, *Process Mining - Data Science in Action, Second Edition.* Springer, 2016. [Online]. Available: https://doi.org/10.1007/978-3-662-49851-4

[2] J. Carmona, B. F. van Dongen, A. Solti, and M. Weidlich, *Conformance Checking - Relating Processes and Models.* Springer, 2018. [Online]. Available: https://doi.org/10.1007/978-3-319-99414-7

[3] A. Adriansyah, "Aligning observed and modeled behavior," 2014.

[4] M. Boltenhagen, T. Chatain, and J. Carmona, "A discounted cost function for fast alignments of business processes," in *Business Process Management - 19th International Conference, BPM 2021, Rome, Italy, September 06-10, 2021, Proceedings*, ser. Lecture Notes in Computer Science, A. Polyvyanyy, M. T. Wynn, A. V. Looy, and M. Reichert, Eds., vol. 12875. Springer, 2021, pp. 252–269. [Online]. Available: https://doi.org/10.1007/978-3-030-85469-0_17

[5] S. Cheikhrouhou, S. Kallel, N. Guermouche, and M. Jmaiel, "The temporal perspective in business process modeling: a survey and research challenges," *Service Oriented Computing and Applications*, vol. 9, pp. 75–85, 2014.

[6] J. Eder, E. Panagos, and M. Rabinovich, "Time constraints in workflow systems," in *CAiSE*, 1999.

[7] A. Nguyen, S. Chatterjee, S. Weinzierl, L. Schwinn, M. Matzner, and B. Eskofier, *Time Matters: Time-Aware LSTMs for Predictive Business Process Monitoring*, 03 2021, pp. 112–123.

[8] W. Aalst, H. Schonenberg, and M. Song, "Time prediction based on process mining," *Inf. Syst.*, vol. 36, pp. 450–475, 04 2011.

[9] A. Rogge-Solti, R. Mans, W. M. P. van der Aalst, and M. Weske, "Repairing event logs using timed process models," in *On the Move to Meaningful Internet Systems: OTM 2013 Workshops - Confederated International Workshops: OTM Academy, OTM Industry Case Studies Program, ACM, EI2N, ISDE, META4eS, ORM, SeDeS, SINCOM, SMS, and SOMOCO 2013, Graz, Austria, September 9 - 13, 2013, Proceedings*, ser. Lecture Notes in Computer Science, Y. T. Demey and H. Panetto, Eds., vol. 8186. Springer, 2013, pp. 705–708. [Online]. Available: https://doi.org/10.1007/978-3-642-41033-8_89

[10] R. Conforti, M. L. Rosa, A. H. M. ter Hofstede, and A. Augusto, "Automatic repair of same-timestamp errors in business process event logs," in *Business Process Management - 18th International Conference, BPM 2020, Seville, Spain, September 13-18, 2020, Proceedings*, ser. Lecture Notes in Computer Science, D. Fahland, C. Ghidini, J. Becker, and M. Dumas, Eds., vol. 12168. Springer, 2020, pp. 327–345. [Online]. Available: https://doi.org/10.1007/978-3-030-58666-9_19

[11] W. M. P. van der Aalst and L. F. R. Santos, "May I take your order? - on the interplay between time and order in process mining," in *Business Process Management Workshops - BPM 2021 International Workshops, Rome, Italy, September 6-10, 2021, Revised Selected Papers*, ser. Lecture Notes in Business Information Processing, A. Marrella and B. Weber, Eds., vol. 436. Springer, 2021, pp. 99–110. [Online]. Available: https://doi.org/10.1007/978-3-030-94343-1_8

[12] T. Aura and J. Lilius, "Time processes for time petri-nets," in *Application and Theory of Petri Nets 1997, 18th International Conference, ICATPN '97, Toulouse, France, June 23-27, 1997, Proceedings*, ser. Lecture Notes in Computer Science, P. Azéma and G. Balbo, Eds., vol. 1248. Springer, 1997, pp. 136–155. [Online]. Available: https://doi.org/10.1007/3-540-63139-9_34


## Appendix

We will detail the proofs of the theorems and lemmas stated above.

### A. Order

*Proof.* We assume that there is at least one mixed move at each position, adding zero moves as needed. We first observe the following:

$$\forall i < j : (s, d, i)(s', d', j)(\gamma) = \gamma' = (s', d', j)(s, d, i)(\gamma)$$

Where

$$\forall 1 \leq k \leq n : \gamma'_k = \begin{cases} \gamma_k & k < i \\ \gamma_k + s + d & k = i \\ \gamma_k + d & i < k < j \\ \gamma_k + s' + d + d' & k = j \\ \gamma_k + d + d' & k > j \end{cases}$$

That is, on linear timed traces, mixed moves are commutative. Clearly permuting the moves of a run does not change its cost, so we can assume any sequence of moves to be in increasing order of positions without loss of generality.

Now we can write $m'$ as follows :

$$m' = (s_{11}, d_{11}, 1) \dots (s_{ij}, d_{ij}, i) \dots (s_{nk_n}, d_{nk_n}, n)$$

Where for each $i$, $k_i \geq 1$ is the number of mixed moves at position $i$.

$$cost(m') = \sum_{i=i}^{n} (\sum_{j=1}^{k_i} |s_{ij}| + |d_{ij}|)$$

Define $s_i = \sum_{j=1}^{k_i} s_{ij}$, $d_i = \sum_{j=1}^{k_i} d_{ij}$, giving us the chronological run

$$m = (s_1, d_1, 1) \dots (s_i, d_i, i) \dots (s_n, d_n, n)$$

Now, by the triangle inequality we know that :

$$\forall i : \sum_{j=1}^{k_i} |s_{ij}| + |d_{ij}| \geq |\sum_{j=1}^{k_i} s_{ij}| + |\sum_{j=1}^{k_i} d_{ij}| = cost(s_i, d_i, i)$$

Thereby giving us that

$$cost(m) \leq cost(m')$$

As for the reverse chronological sequence, by commutativity we can simply reverse $m$ and obtain $m_R$, a reverse chronological sequence of moves that does the same transformation and as reversing the sequence preserves cost,

$$cost(m_R) = cost(m) \leq cost(m')$$

□

### B. Co-operation

*Proof.* We first prove that there is a co-operative and chronological minimal cost sequence of moves aligning any two linear timed traces. Given any minimal cost sequence of moves aligning $\gamma$ to $\sigma$, by Lemma 2 we know that there is a chronological sequence that is also minimal cost, call it $m'$.

Assume $|\gamma| = |\sigma| = n$.

We proceed by induction on the first index $k$ at which $m' = (s_1, d_1, 1) \dots (s_n, d_n, n)$ uses a non co-operative move.

**Base case** : $k = n$.

This gives us

$$s_n d_n < 0 \implies |s_n + d_n| < |s_n| + |d_n|$$
$$\implies m = m'|_{n-1}(0, s_n + d_n, n)$$

costs less than $m'$, aligns the words and is co-operative.

**Induction hypothesis** : Suppose we've demonstrated that there is a sequence of moves $m''$ that aligns $\gamma$ to $\sigma$, is co-operative for the first $k - 1 < n - 1$ steps, is non co-operative at step $k$ and $cost(m'') \leq cost(m)$

**Claim** : There is a sequence of moves $m$ that aligns $\gamma$ to $\sigma$, is co-operative for at least the first $k$ steps and

$$cost(m) \leq cost(m'') \leq cost(m')$$

**Proof** : Let $m'' = (s_1, d_1, 1) \dots (s_n, d_n, n)$.

We first define $m_i = m''|_{k-1}$, $m_f = (s_{k_2}, d_{k+2}, k + 2) \dots (s_n, d_n, n)$, thereby giving us

$$m'' = m_i(s_k, d_k, k)(s_{k+1}, d_{k+1}, k + 1)m_f$$

We know that $s_k d_k < 0$. This can be split into the following cases :

**Case 1** : $|d_k + s_k| = |d_k| - |s_k|$

We define the following aligning, chronological run that is co-operative up to (and including) the $k$th step :

$$m = m_i(0, d_k + s_k, k)(s_{k+1}, d_{k+1} - s_k, k + 1)m_f$$

$$cost(m'') - cost(m)$$

$$= |s_{k+1}| + |d_{k+1}| + |s_k| + |d_k| - (|d_k + s_k| + |s_{k+1}| + |d_{k+1} - s_k|)$$

$$\geq (|s_{k+1}| + |d_{k+1}| - |s_{k+1}| + |d_{k+1}|) + (|s_k| + |d_k| - |d_k + s_k| - |s_k|) \geq 0$$

**Case 2** : $|d_k + s_k| = |s_k| - |d_k|$

We define the following aligning, chronological run that is co-operative up to (and including) the $k$th step :

$$m = m_i(d_k + s_k, 0, k)(s_{k+1}, d_k + d_{k+1}, k + 1)m_f$$

$$cost(m'') - cost(m)$$

$$= |s_{k+1}| + |d_{k+1}| + |s_k| + |d_k| - (|s_k + d_k| + |s_{k+1}| + |d_{k+1} + d_k|)$$

$$\geq |d_{k+1}| + |s_k| + |d_k| - (|s_k + d_k| + |d_{k+1}| + |d_k|) \geq 0$$

By induction we can hence conclude that there is a chronological sequence of moves $m$ that is co-operative

throughout and has lower cost than $m'$, i.e, if $m'$ is minimal cost so is $m$.

Now, $m_R$ is also co-operative throughout as co-operation is a property of the individual moves constituting the sequence, and $m_R$ has exactly the same moves just in the reverse order, so there is also a reverse chronological minimal cost sequence of moves aligning $\gamma$ to $\sigma$.

□

*Proof.* Say $m' = (0, d'_n, n) \dots (s'_1, d'_1, 1)$. Let $i$ be an index at which the stamp move is not cross co-operative, i.e, $s_i \cdot d_{i+1} > 0$.

Consider a run $m''$ that is identical to $m'$, except for the moves it plays at positions $i$ and $i+1$.

Let $m''$ perform the move $(0, d_i + s_i, i)$ at position $i$ and $(s_{i+1}, d_{i+1} - s_i, i+1)$ for the move at position $i+1$ if $|s_i| < |d_i|$ and $(s_i - d_{i-1}, d_i + d_{i+1}, i)$ at position $i$ and $(s_{i+1}, 0, i+1)$ for the move at position $i+1$ otherwise.

Clearly, the appropriate one of either of these is cross co-operative at position $i$, while also staying co-operative and reverse chronologically aligning $\gamma$ to $\sigma$, and costs $\min(|s_i|, |d_{i-1}|)$ less that $m'$.

Doing this procedure for at most $n$ times will result in a run with every stamp move being cross co-operative and with lower cost than $m'$, that is, our required $m$. □

### C. Stability

Before we proceed with the proof of this result, we prove a small technical lemma that will help us in the proof of Lemma 6. The motivation behind the following result is essentially that the mixed distance $d_N$ respect an intuitive notion of closeness, once two words have been aligned up till the penultimate place, the one whose last duration since firing lands it closer to the desired target will naturally be the easier one to align. Put another way, if two words, aligning to a target word, are identical to each other in all respects but the last timestamp, then the one whose last flow value is closer to the desired target flow value, is the word that is over closer to the target under $d_N$ between the two.

**Lemma 10.** *Given two timing functions*

$$\gamma^x = (\gamma_1, \gamma_2, \dots \gamma_{n-1}, \gamma_{n-1} + x)$$

$$\gamma^y = (\gamma_1, \gamma_2, \dots \gamma_{n-1}, \gamma_{n-1} + y)$$

*Both aligning to* $\sigma = (\sigma_1, \dots, \sigma_n)$*, such that* $|x - (\sigma_n - \sigma_{n-1})| \leq |y - (\sigma_n - \sigma_{n-1})|$ *then*

$$d_N(\sigma, \gamma^x) \leq d_N(\sigma, \gamma^y)$$

*Proof.* Consider any run $m^y$ aligning $\gamma^y$ to $\sigma$. We claim there is a run $m^x$ aligning $\gamma^x$ to $\sigma$ such that $cost(m^x) \leq cost(m^y)$.

This proves the above lemma as then the minimal cost run aligning $\gamma^x$ to $\sigma$ would have cost at most $m^x$ corresponding to the minimal cost run aligning $\gamma^y$ to $\sigma$, so

$$d_N(\sigma, \gamma^x) \leq d_N(\sigma, \gamma^y)$$

Now, we can assume without loss of generality by Lemmas 2 and 3 that $m^y$ is chronological and co-operative, say $m^y = (s_1, d_1, 1) \dots (0, d_n, n)$.

We construct $m^x = (s_1, d_1, 1)(s_2, d_2, 2) \dots (s'_{n-1}, d'_{n-1}, n-1)(0, d'_n, n)$ differing at most at the last two mixed moves. As $m^x|_{n-2} = m^y|_{n-2}$ we can also conclude that $s_{n-1} + d_{n-1} = s'_{n-1} + d'_{n-1}$ by chronology, so with co-operation we can conclude that $cost(m^y) - cost(m^x) = |d_n| - |d'_n|$.

Hence, the problem is reduced to proving that whatever the choice of $d_n$, there is a choice of $d'_n$ that fully aligns $\gamma^x$ and has atmost equal magnitude.

Now, $|d_n| = |\sigma_{n-1} + y - \sigma_n|$ as $m^y|_{n-1}$ fully corrected up till the penultimate place, and similarly $|d'_n| = |\sigma_{n-1} + x - \sigma_n|$, hence, by definitions of $x$ and $y$, we have the result. □

Now, we can proceed with the proof of Lemma 6.

*Proof.* Suppose we seek to align the linear timed trace $\gamma$ to $\sigma$, and consider a minimal cost, reverse chronological, co-operative and cross co-operative sequence of moves $m'$ that aligns $\gamma$ to $\sigma$. We know $m'$ exists by Lemmas 3 and 4. On the other hand, consider the unique stable, co-operative, reverse chronological sequence of moves $m$ that aligns $\gamma$ to $\sigma$, that exists by Lemma 5.

We proceed by way of induction. In the base case of words of length 1, the claim is clearly true as the minimal cost aligning move would just be the minimum delay move, which is also the unique stable run. For words of length 2, i.e, $\gamma_1\gamma_2$ aligning to $\sigma_1\sigma_2$, a reverse chronological co-operative run on the flow function is of the following form :

$$m' : f_\gamma \xmapsto{(0,d_2)} (f_\gamma(1), f_\gamma(2) + d_2)$$

$$\xmapsto{(s_1,d_1)} (f_\gamma(1) + s_1 + d_1, f_\gamma(2) - s_1 + d_1) = f_\sigma$$

By reverse chronology and co-operation, we have the following equation :

$$cost(m') = |d_2| + |s_1 + d_1| = |f_\sigma(2) - f_\gamma(2) + s_1| + |f_\sigma(1) - f_\gamma(1)|$$

Clearly the only variable in the cost is the value of $s_1$, and the stable run $m$ is the run that precisely minimises the value $|f_\sigma(2) - f_\gamma(2) + s_1|$ while maintaining co-operation, so the claim holds.

Now, let the length of $\gamma$ be $n > 2$, and suppose the claim holds for all words of length less than $n$. We claim that $cost(m) \leq cost(m')$.

The two runs must diverge at some point, as otherwise their cost is equal and the claim is obvious. There are only two main cases of interest :

**Case 1.** The first move is the same, but the second moves are different (i.e the mixed move played at the second right-most position of the trace).

**Case 2.** The first moves of the two sequences $m$ and $m'$ (i.e the delay played on the right-most position of the trace) are different.

If the first move where the sequences diverge are later, say at some position $i$ in the trace, then we can restrict our attention to the $i+1$ length prefix of the word and the appropriate truncations of $m$ and $m'$, and by induction hypothesis we are done. So, we focus on the above cases henceforth.

**Case 1.** Let $m = (0, d_n, n)(s_{n-1}, d_{n-1}, n-1)\dots(s_1, d_1, 1)$ and
$m' = (0, d'_n, n)(s'_{n-1}, d'_{n-1}, n-1)\dots(s'_1, d'_1, 1)$

As both these runs are reverse chronological, we obtain the following equalities:

$$\forall i > 1 : s_i + d_i - s_{i-1} = f_\sigma(i) - f_\gamma(i) = s'_i + d'_i + s'_{i-1}$$

Setting $i$ to $n$ gives $d_n - s_{n-1} = d'_n - s'_{n-1} = f_\sigma(n) - f_\gamma(n)$, which implies that since $d_n = d'_n$, $s_{n-1} = s'_{n-1}$.

Now consider $\gamma_s = (s_{n-1}, 0, n-1)\gamma|_{n-1}$, and
$m_s = (0, d_{n-1}, n-1)(s_{n-2}, d_{n-2}, n-2)\dots(s_1, d_1, 1)$ and
$m'_s = (0, d'_{n-1}, n-1)(s'_{n-2}, d'_{n-2}, n-2)\dots(s_1, d_1, 1)$.

Clearly $m_s$ is the stable run aligning $\gamma_s$ to $\sigma|_{n-1}$ and $m'_s$ is another reverse chronological co-operative aligning sequence, and

$$cost(m) = |d_n| + |s_{n-1}| + cost(m_s)$$
$$cost(m') = |d'_n| + |s'_{n-1}| + cost(m'_s)|$$

and by triangle inequality and the stability of $m$,

$$|d_n| + |s_{n-1}| = |d_n - s_{n-1}| = |d'_n - s'_{n-1}| \le |d'_n| + |s_{n-1}|$$

Now, by induction hypothesis on $\gamma_s, m_s, m'_s$ (they're an instance of length $n-1$ words, case 2), we deduce $cost(m_s) \le cost(m'_s)$, by which we can conclude that

$$cost(m) \le cost(m')$$

**Case 2.** Let $m = (0, d_n, n)(s_{n-1}, d_{n-1}, n-1)\dots(s_1, d_1, 1)$ and $m' = (0, d'_n, n)(s'_{n-1}, d'_{n-1}, n-1)\dots(s'_1, d'_1, 1)$

As both these runs are reverse chronological, we obtain the following equalities:

$$\forall i > 1 : s_i + d_i - s_{i-1} = f_\sigma(i) - f_\gamma(i) = s'_i + d'_i + s'_{i-1}$$

Setting $i$ to $n$ gives $d_n - s_{n-1} = d'_n - s'_{n-1} = f_\sigma(n) - f_\gamma(n)$, which implies that having performed the first delay and stamp move of each run, one gets the following :

$$f_\gamma \xmapsto{(0,d,n)(s,0,n-1)} (f_\gamma(1), \dots f_\gamma(n-1) + s, f_\sigma(n))$$
$$f_\gamma \xmapsto{(0,d',n)(s',0,n-1)} (f_\gamma(1), \dots f_\gamma(n-1) + s', f_\sigma(n))$$

Where $s = s_{n-1}$, $d = d_n$, $s' = s'_{n-1}$, and $d' = d'_n$.

Now, let $e_n = f_\sigma(n) - f_\gamma(n)$, and $e_{n-1} = f_\sigma(n-1) - f_\gamma(n-1)$ Now, by stability of $m$, we have the following definition of $s$ :

$$s = \begin{cases} 0 & e_n \cdot e_{n-1} \ge 0 \\ e_{n-1} & \text{Else If } |e_{n-1}| < |e_n| \\ -e_n & \text{Otherwise} \end{cases}$$

By the above definition, we see that $s = \operatorname{argmin}_{x\cdot(e_n - x)\ge 0} |e_{n-1} - x|$.

Now by cross co-operation, we know that $s' \cdot (e_n - s') \ge 0$, so this means

$$|e_{n-1} - s| \le |e_{n-1} - s'|$$
$$\implies |f_\sigma(n-1) - (f_\gamma(n-1) + s)| \le |f_\sigma(n-1) - (f_\gamma(n-1) + s')|$$

Now, consider the words $\gamma_s = (\gamma_1, \dots \gamma_{n-2}, \gamma_{n-1} + s)$ and $\gamma'_s = (\gamma_1, \dots \gamma_{n-2}, \gamma_{n-1} + s')$ both aligning to $\sigma|_{n-1}$. We can apply Lemma 10 to these, and deduce that

$$d_N(\gamma_s, \sigma|_{n-1}) \le d_N(\gamma'_s, \sigma|_{n-1})$$

Now, by the induction hypothesis, as the rest of $m$ would have been the stable minimal cost run aligning $\gamma_s$ to $\sigma|_{n-1}$, along with stability of $m$, we have the following :

$$cost(m) = |s| + |d| + d_N(\gamma_s, \sigma|_{n-1})$$
$$\le |d - s| + d_N(\gamma'_s, \sigma|_{n-1}) \le cost(m')$$

And hence we are done.

□

### D. Computing $d_N$ and Alignments

*Proof.* As $(CN, p)$ is linear, we can consider the event set $E$ to be totally ordered by $G$, giving us the list $\{e_1, e_2, \dots e_n\}$ and their respective static interval constraints $\forall i \le n : SI(p(e_i)) = [a_i, b_i]$. In this setting, clearly,

$$f_\gamma(i) = \operatorname*{argmin}_{x \in [a_i, b_i]} |x - f_\sigma(i)|$$

We claim that for all $\alpha \in \mathcal{L}(N)$ i.e $\forall \alpha : f_\alpha(i) \in [a_i, b_i]$,

$$d_N(\sigma, \gamma) \le d_N(\sigma, \alpha)$$

By Theorem 8 we shall henceforth implicitly equate the cost of the stable run aligning a word to the mixed moves distance to the word. Let $m, m'$ be the stable runs aligning $\gamma, \alpha$ respectively to $\sigma$, where:

$$m = (0, d_n, n)(s_{n-1}, d_{n-1}, n-1)\dots(s_1, d_1, 1)$$
$$m' = (0, d'_n, n)(s'_{n-1}, d'_{n-1}, n-1)\dots(s'_1, d'_1, 1)$$

In addition, we know that $cost(m) = \sum_{i=1}^n (|s_i| + |d_i|)$, and $cost(m') = \sum_{i=1}^n (|s'_i| + |d'_i|)$.

We break the run up after each stamp move, but not delay move, at a particular position has been performed, as below:

$$\forall i > 0 : c_i = |s_{i-1} + |d_i|, c'_i = |s'_{i-1}| + |d'_i|$$

Where we let $s_0 = s'_0 = 0$ for notation's sake. Clearly $cost(m) = \sum_{i=1}^n c_i$ and $cost(m') = \sum_{i=1}^n c'_i$

We wish to prove that

$$cost(m) \le cost(m')$$

In fact, we shall prove something stronger. We define the notion of the disadvantage of $m$, which denotes in effect the

amount of distance it might have to catch up in aligning due to $m'$ having done a costlier move just prior.

$$dis(i) = max(0, c'_i - c_i)$$

Note of course, that each $c_i$, $c'_i$ and $dis(i)$ is nonnegative.

Now, we claim the following :

$$\forall k > 0 : \sum_{i=k}^{n} c_i + dis(k) \leq \sum_{i=k}^{n} c'_i$$

If we prove this, we will be done, as the equation for $k = 1$ proves that $m$ has overall lower cost than $m'$.

Let us proceed by induction on $n - k$.

For the base case, $k = n$, so the equation we seek is that $c_n + dis(n) \leq c'_n$, which is clear as $dis(n) = max(0, c'_n - c_n)$.

Now, say the claim holds for all $k > K$ for some $K < n$. We seek to now prove that

$$c_K + \sum_{i=K+1}^{n} c_i + dis(K) \leq c'_K + \sum_{i=K+1}^{n} c'_i$$

Now, if $c_K \leq c'_K$ this is clear as $c_K + dis(K) = max(c_K, c'_K)$. On the other hand, suppose $c_K > c'_K$.

By the induction hypothesis, we know that

$$\sum_{i=K+1}^{n} c_i + dis(K+1) \leq \sum_{i=K+1}^{n} c'_i$$

So if we show that $c_K - c'_K \leq dis(K+1)$, we are done.

Now, by reverse chronology, co-operation and cross co-operation of $m$ and $m'$, and the definition of $\gamma$, we know the following inequality holds :

$$|s_K| + c_K = |s_K + d_K - s_{K-1}| = |f_\sigma(K) - f_\gamma(K)|$$

$$\leq |f_\sigma(K) - f_\alpha(K)| = |s'_K + d'_K - s'_{K-1}| = |s'_K| + c'_K$$

So, $c_K > c'_K \implies |s_K| < |s'_K|$, and moreover,

$$c_K - c'_K \leq |s'_K - s_K|$$

Now, by stability of $m$, we know that $s_K$ was defined as below :

$$s_K = \begin{cases} 0 & e_K \cdot e_{K+1} \geq 0 \\ e_K & \text{Else If } |e_K| < |e_{K+1}| \\ e_{K+1} & \text{Otherwise} \end{cases}$$

Where $e_K = (f_\sigma(K) - f_\gamma(K))$ and $e_{K+1} = f_\sigma(K+1) - f_\gamma(K+1) - s_{K+1}$.

We proceed to analyse the possibilities based on the various possible stable choices of $s_K$ depending on the state of the run so far.

Suppose firstly that $s_K = 0$. There are a number of ways this could hold. By choice of $\gamma$,

$$(f_\sigma(K) - f_\gamma(K)) \cdot (f_\sigma(K+1) - f_\gamma(K+1)) > 0$$

$$\implies (f_\sigma(K) - f_\alpha(K)) \cdot (f_\sigma(K+1) - f_\alpha(K+1)) > 0 \implies s'_K = 0$$

But $|s'_K| > |s_K| \geq 0$ so this first subcase is not possible.

Secondly, if

$$(f_\sigma(K) - f_\gamma(K)) = 0 \implies c_K = 0 > c'_K$$

Which is also not possible.

Thirdly, if

$$(f_\sigma(K+1) - f_\gamma(K+1) - s_{K+1} = 0 \implies d_{K+1} = 0$$

Now, $d_{K+1} = 0$ and $|s'_K| > |s_K|$ implies

$$dis(K+1) = |s'_K| + |d'_{K+1}| - |s_K| \geq |s'_K| - |s_K| \geq c_K - c'_K$$

So in this case the claim holds as needed.

Otherwise, suppose $s_K = f_\sigma(K) - f_\gamma(K)$. In this case, $c_K$ is once again equal to zero, which is not possible as by assumption $c_K > c'_K \geq 0$.

Now, the last possibility is that $s_K = f_\gamma(K+1) - f_\sigma(K+1) - s_{K+1}$. This means that once $s_K$ is played, the $K+1$ position is perfectly aligned, so $d_{K+1} = 0$. Once again, this means

$$dis(K+1) = |s'_K| + |d'_{K+1}| - |s_K| \geq |s'_K| - |s_K| \geq c_K - c'_K$$

So in this case also, the claim holds as needed.

Hence, we see that in every possible case, the claim holds for $k = K$ as well, so by induction we see that

$$\sum_{i=1}^{n} c_i + dis(K+1) \leq \sum_{i=1}^{n} c'_i$$

From which we can conclude that

$$cost(m) = \sum_{i=1}^{n} c_i) \leq \sum_{i=1}^{n} c'_i = cost(m')$$

Hence, the result holds, and $\gamma$ is an optimal word of $\mathcal{L}(N)$ to align $\sigma$ to.

$\square$